Embedded Software and Deterministic Multitasking

By: >_ terminal gremlin (c-lydia ^-^)

On a desktop operating system, the kernel's preemptive scheduler transparently balances hundreds of background processes. On microcontroller, the CPU executes exactlu what is told, sequentially and without intervention.

Using `delay()` blocks the CPU, although is harmless, it completely paralyzes a microcontroller by burning millions of clock cycles executing empty loops, unable to read an incoming button press, poll a communication bus, or service other hardware peripherals. However, we need multiple tasks being executed concurrently on the real robot with predictable timing.

Blocking and Non-Blocking Execution

The Problem with Blocking Delay (`delay()`)

Microcontroller software framework provide the built-in function `delay(ms)` as an easy way to pause execution. When firmware invokes `delay()`, the microcontroller enters a tight, synchronous spin loop

```
// Conceptual disassembly of a synchronous blocking delay
void blocking_delay(uint32_t ms) {
    uint32_t start_tick = get_system_ticks();

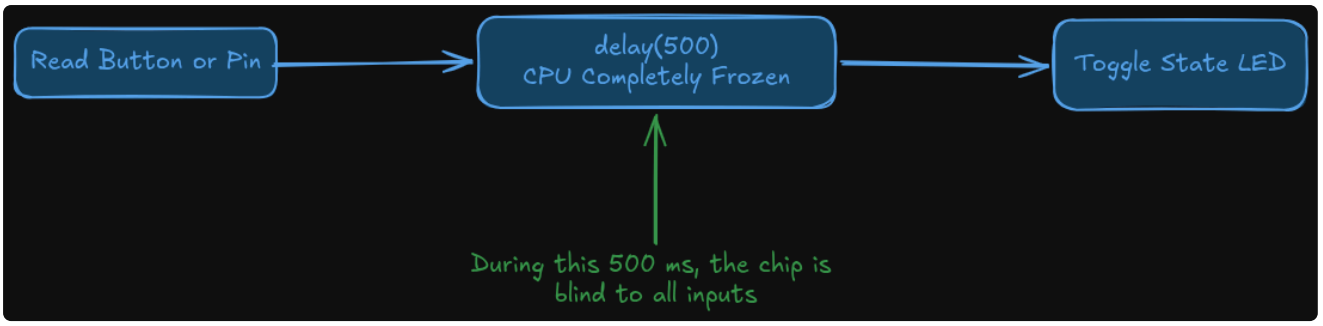
    while (get_system_ticks() - start_tick < ms * TICKS_PER_MS) {
        // CPU burns clock cycles executing No-Operation (NOP)
instructions
        // Main thread is frozen; zero other user tasks can execute
    }
}
```

During this pause, the CPU core is completely monopolized. If a program calls `delay(500)`:

- **Inputs are ignored:** if an operator presses an emergency stop push-button on a GPIO pin, the microcontroller cannot detect the failing edge
- **Serial buffers overflow:** if an external computer streams commands over UART, the microcontroller fails to read bytes from the hardware receive buffer, leading to buffer

overruns and dropped packets

- **Outputs freeze:** other digital pins cannot toggle, preventing the cconcurrent blinking of status indicators or scheduled polling of peripheral sensors



Timestamp Delta Subtraction

To allow multiple operations to run concurrently without freezing the CPU, embedded firmware uses an **event-driven polling architecture** driven by free-running hardware timers.

In the ESP32 Arduino framework, the internal hardware counter provides two free-running clock functions:

- **`millis()`**: returns the number of milliseconds elapsed since the chip booted as a 32-bit unsigned integer (`uint32_t`)
- **`micros()`**: returns the number of microseconds elapsed since the chip booted as a 32-bit unsigned integer (`uint32_t`)

Instead of halting the processor, the firmware allows the `loop()` function to cycle as fast as possible (often tens of thousands of times per second). On every iteration, the firmware checks whether a target time interval has elapsed by evaluating the mathematical difference

$$\Delta t = t_{current} - t_{previous}, \Delta t \geq T_{interval}$$

- If $\Delta t < T_{interval}$, the task is not ready; the CPU skips the task immediately (in less than $1\mu s$) and proceeds to check other tasks
- If $\Delta t \geq T_{interval}$, the interval has elapsed; the CPU executes the task, updates $t_{previous}$, and continues cycling

Periodic Scheduling: Relative Reset and Absolute Grid Anchoring

When updating the stored timestamp after a task executes, the update method directly influences timing accuracy

Relative Reset (`previous_time = current_time`)

```
if (current_time - previous_time >= INTERVAL) {
    previous_time = current_time; // Reset relative to the moment of
execution
    execute_task();
}
```

- **Behavior:** if a task with an interval of $10ms$ is delayed by other code and fires at $12ms$, setting `previous_time = current_time` anchors the next cycle to $12ms$. The next execution will occur at $12 + 10 = 22ms$ instead of the ideal $20ms$
- **Consequence:** the $12ms$ delay is permanently incorporated into all future cycles. Over minutes and hours, these micro-delays accumulate, resulting in **cumulative timebase dift** where the task executes fewer total cycles than expected

Absoulte Grid Anchoring (`previous_time += INTERVAL`)

```
if (current_time - previous_time >= INTERVAL) {
    previous_time ++ INTERVAL; // Advance epoch by exact fied interval
    execute_task();
}
```

- **Behavior:** if the task scheduled for $10ms$ executes late at $12ms$, adding `INTERVAL` ($10ms$) updates `previous_time` to exactly $10ms$. The next cycle will still evaluate against the original grid target ($20ms$)
- **Advantage:** temporary execution laatency does not shift the long-term sechdule. Over time, the average task frequency remains exact with **zero cumulative drift**. This is the standard method for precision periodic scheduling

Unsigned Integer Overflow: Rollover Invariance

A frequent convern in embedded timing is what happens when the 32-bit hardware timer runs out of bits and rolls over to zero.

A 32-bit unsigned integer (`uint32_t`) can hold values from 0 up to $2^{32} - 1 = 4,294,967,295$

- For `micros()` , the counter rolls over every

$$t_{overflow} = \frac{4,294,967,295\mu s}{1,000,000\mu s/s} \approx 4249.97s \approx 71.58mn$$

- For `millis()`, the counter rolls over every

$$t_{overflow} = \frac{4,294,967,295ms}{1,000ms/s \cdot 86,400s/day} \approx 49.21days$$

Why Forward Addition Fails

Native implementations often try to pre-calculate a future deadline using addition

```
// INCORRECT AND DANGEROUS IMPLEMENTATION
uint32_t deadline = millis() + INTERVAL;

void loop() {
     // BREAKS COMPLETELY AT ROLLOVER
     if (millis() >= deadline) {
          deadline = millis() + INTERVAL;
          execute_task();
     }
}
```

- **The Failure:** when `millis()` reaches $4,294,967,290$ and `INTERVAL` is 10, the calculation $4,294,967,290 + 10$ overflows in 32-bit math to 4. Because $4,294,967,290 \geq 4$, the `if` condition evaluates to `true` instantly. Worse, once the time overflow past zero, `0 >= 4` evaluates to `false` and remain `false` until the timer reaches 4, locking up the task for nearly 50 days

The subtraction paradigm is immune to rollover issues

```
// CORRECT IMPLEMENTATION
if ((uint32_t) (current_time - previous_time) >= INTERVAL) {
     previous_time += INTERVAL;
     execute_task();
}
```

In C and C++, arithmetic on unsigned integers is defined to obey modular arithmetic modulo 2^N (where $N = 32$). The subtraction computes

$$\Delta t = t_{current} - t_{previous} \pmod{2^{32}}$$

Let `previous_time` be recorded just before rollover at $t_{previous} = 4,294,967,290$
Let `current_time` be sampled just after rolleover at $t_{current} = 5$

The true elapsed physical time is

$$\Delta t_{physical} = (4,294,967,296 - 4,294,967,290) + 5 = 6 + 5 = 11ms$$

Now, evaluate the subtraction in unsigned 32-bit math

$$5 - 4,294,967,290 = -4,294,967,285$$

Mapping $-4,294,967,285$ into the unsigned modular space modulo 2^{32}, we have

$$-4,294,967,285 + 2^{32} = -4,294,967,285 + 4,294,967,296 = 11$$

The unsigned calculation yields **exactly** 11, identical to physical elapsed time. As long as the execution intervals is less than $2^{32} - 1$ ticks (471.6$ minutes for `micros()`, 49.7 days for `millis()`), timestamp subtraction yields mathematically exact results across the rollover boundary with no special-case code required.

Execution Budgets, CPU Utilization, and Software Jitter

To guarantee that an embedded system remains responsive, you must track execution budgets and timing quality

Execution Budget and Period

For a task running at frequency f_s, we have

$$T_{period} = \frac{1}{f_s}$$

- At $100Hz. T_{period} = 10ms = 10.00\mu s$
- At $10Hz, T_{period} = 100ms = 100,000\mu s$

CPU Utilization (Load Factor ρ)

If a task takes T_{exec} microseconds to execute, its CPU load factor is

$$\rho = \frac{T_{exec}}{T_{period}} \cdot 100$$

- **Healthy Headroom ($ho \leq 0.2$):** the task consumes a modest slice of the CPU, leaving ample time for communication tasks and background polling
- **Overrun ($T_{exec} > T_{period}$):** a deadline overrun occurs. The tasl cannot complete before its next cycle is due, causing the scheduler to stall and miss future time slots

Timing Jitter (J)

Jitter represents the variation between the ideal scheduled time and the actual moment of execution

$$J[k] = |T_{actual}[k] - T_{period}|$$

In cooperative multitasking, jitter occurs when one task runs slightly longer than usual or when a hardware interrupt briefly pauses the main loop. Keeping individual tasks short and avoiding blocking delays minimizes jitter across all tasks.

Cooperative Multitasking and Task Scheduling Architecture

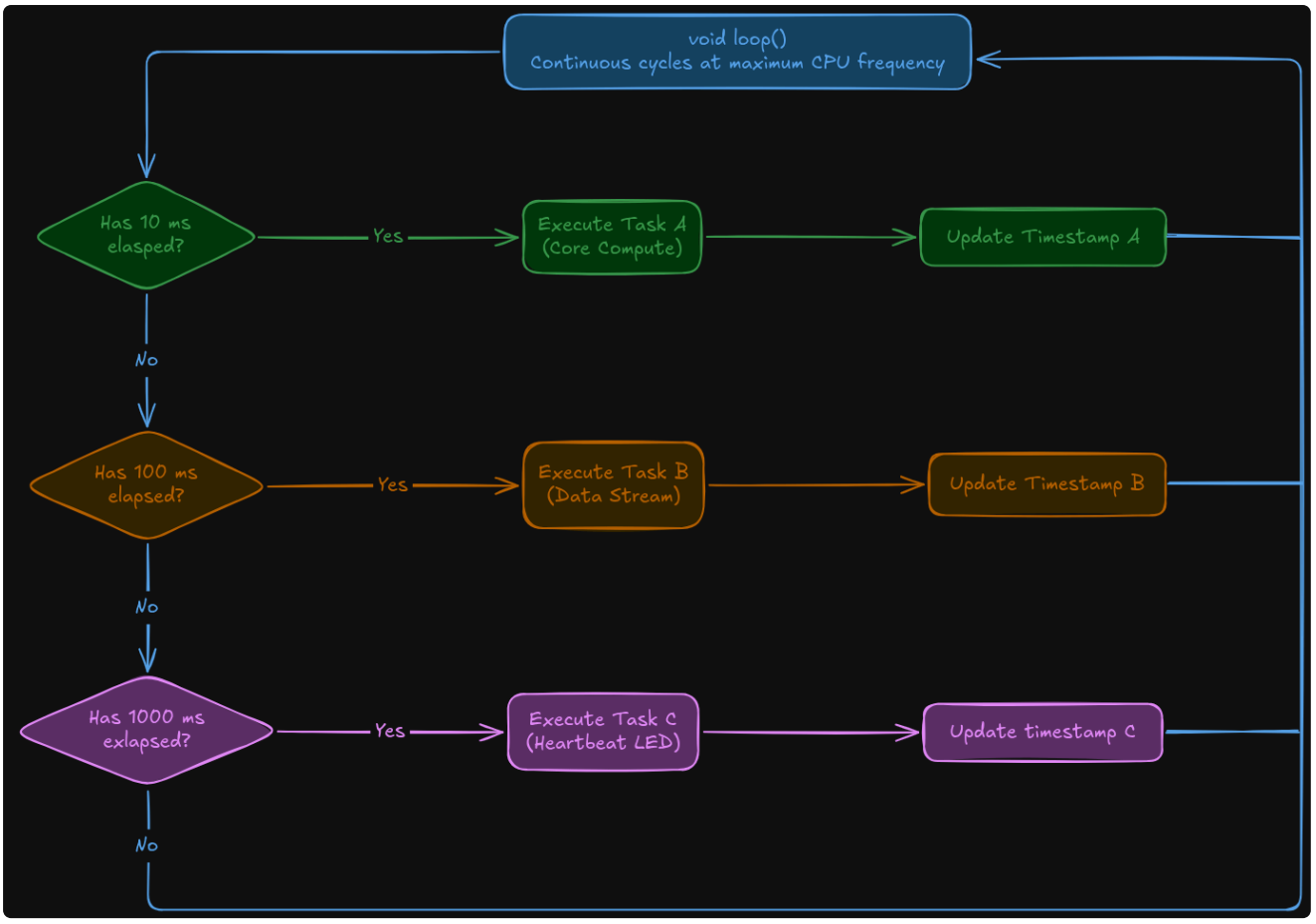
When an embedded system must perform several independent tasks at different rates, cooperative multitasking provides a lightweight, robust alternative to a complex Real-Time Operating System (RTOS).

Principles of Cooperative (Non-Preemptive) Multitasking

In cooperative multitasking:

- There is no underlying OS kernel pausing or preempt running functions
- Each task is structured as a quick, non-blocking function that executes its work and immediately returns control to the main loop
- The system relies on each task being well-behaved (no single task may hog the CPU or execute long delays)

Cooperative Non-Blocking Task Cycle



1. Every task function must execute and return in a fraction of its total period ($T_{\text{exec}} \ll T_{\text{period}}$)
2. Never call `delay()` or long blocking `while()` loops inside any periodic task

Structuring Multi-Rate Periodic Tasks in C++

To keep firmware clean, modular, and easy to maintain inside a single `.ino` file, tasks can be encapsulated using a lightweight helper struct:

```
struct PeriodicTask {
    uint32_t last_execution_time_us;
    const uint32_t interval_us;
    PeriodicTask(uint32_t interval) : last_execution_time_us(0),
    interval_us(interval) {}

    inline bool is_ready(uint32_t current_time_us) {
        if ((uint32_t)(current_time_us - last_execution_time_us) >=
interval_us) {
            last_execution_time_us += interval_us;
            return true;
```

```
        }

        return false;
    }
};
```

Compute, I/O (Input/Output), and Testing

We must separate high-frequency computational tasks from slow input/output (I/O) routines

- **Computational rountines:** arithmetic, state evaluation, bitwise parsing are fast, running in microseconds
- **I/O routines:** formatting text strings, writing to display screens, sending serial data are slow, taking millisecons

By scheduling slow I/O operations at a lower frequency (e.g., $10Hz$) and keeping high-speed tasks at $100Hz$, the system maintains fast responsiveness without choking communications buses.

Serial Communication (UART) and Bus Throughput

The Universal Asynchronous Receiver-Transmitter (UART) is the primary wired communication interface between a microcontroller and an external debugging computer.

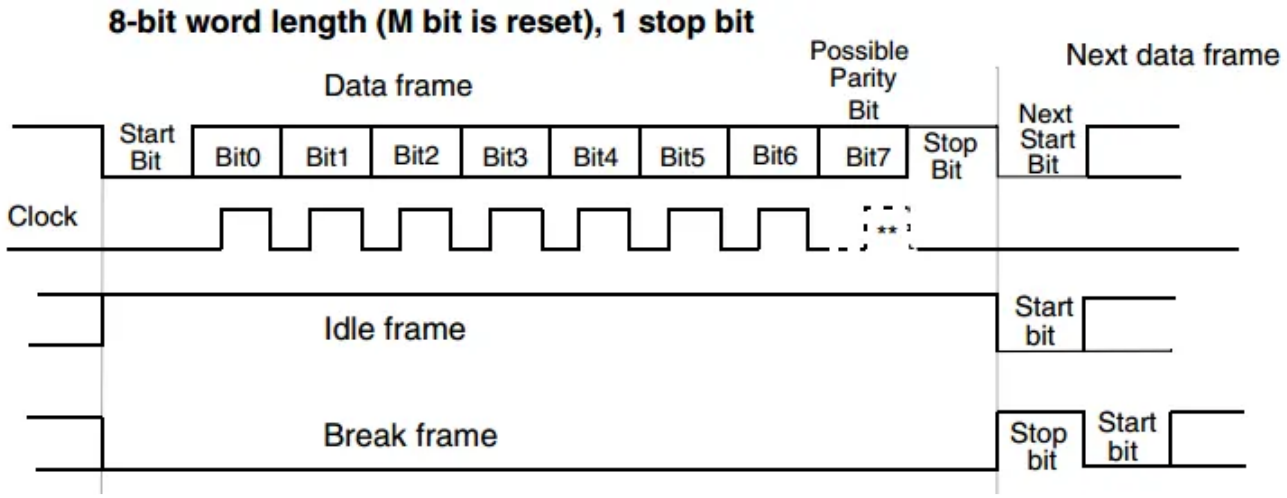
UART Physical Sampling and 8-N-1 Asynchronous Framing

UART is an asynchronous point-to-point serial protocol. Because there is no shared clock line between transmitter and receiver, both devices must agree on a predefined transmission speed: the **baud rate** (bits per second).

Digital logic levels on the physical TX and RX lines are defined as:

- **Mark State (Logic 1):** line sits at idle voltage ($+3.3V$ on the ESP32)
- **Space State (Logic 0):** line drops to Ground ($0.0V$)

Every byte transmitted across UART is wrapped in a standard physical frame. The most universal configuration is **8-N-1** (8 Data Bits, No Parity, 1 Stop Bit)



1. **Start Bit (1 bit):** the line drops from idle HIGH ($3.3V$) to LOW ($0.0V$) for exactly one bit perod, alerting the receiver's hardware shift register
2. **Data Bits (8 bits):** the 8 bits of the payload byte are transmitted sequentially, Least Significant Bit (LSB) first
3. **Stop Bit (1 bit):** the line returns to HIGH ($3.3V$) for at least one bit period to frame the end of the byte and return to idle

$$\text{Ttoal Bits per Byte} = 1\ (\text{Start}) + 8\ (\text{data}) + 0\ (\text{Stop}) = 10 bits/byte$$

Baud Rate, Bit TIme, and Maximum byte Throughput Calculations

For the standard ESP32 debugging speed of $115,200 baud$:

1. **Single Bit Duration (T_{bit}):**

$$T_{bit} = \frac{1}{\text{Baud Rate}} = \frac{1}{115,200 bit/s} \approx 8.6806 \cdot 10^{-6} s \approx 8.681 \mu s$$

2. **Single Byte Duration (T_{byte}):**

$$T_{byte} = 10 \cdot T_{bit} = 10 \cdot 0.6806 \mu s \approx 86.81 \mu s$$

3. **Maximum Byte Throughput (R_{max}):**

$$R_{max} = \frac{115,200 bit/s}{10 bits/byte} = 115,200 bytes/s \approx 11.52 KB/s$$

The Ring Buffer Saturation Trap: How Unchecked Serial Printing Freezes Microcontrollers

A confusing bug in embedded systems occurs when a program that ran smoothly suddenly begins stuttering ot freezing when debug print statements are added.

In the Arduino ESP32 core, `Serial.print()` writes characters into an internal software ring buffer in RAM (typically 128 to 256 bytes in size). As long as the ring buffer has free space:

- `Serial.print()` copies bytes into memory and returns almost immediately (taking less than $1\mu s$)
- The hardware UART peripheral empties this buffer in the background via interrupts

The Overflow Issue

Suppose you attempts to stream data directly inside $100Hz$ task

```
// DANGEROUS PATTERN: streaming heavy data at high frequency
void task_100Hz() {
    // Message legth: ~120 characters per cycle
    Serial.printf("Cycle: %lu, Uptime: %lu, Value1: %.2f, Value2: %.2f,
MetricA: %d, MetricB: %d\n", cycle_count, millis(), v1, v2, mA, mB);
}
```

The transmission bandwidth is

$$\text{Bandwidth Demand} = 120 bytes/cycle \cdot 100 cycles/s = 12,000 bytes/s$$

Comparing the demand with the physical capacity

$$\text{Demand}(12,000 B/s) > \text{Maximum Wire Throughput}(11,520 B/s)$$

Because the program generates data faster than the physical wire can carry it, the transmit ring buffer fills to capacity in less than 200 milliseconds

Once the ring buffer is completely full, **`Serial.print()` can no longer queue data in RAM; it switches to synchronous blocking execution**. The processor is foreced to stop and wait for physical UART bits to shift out over the wire

$$Deltat_{stall} = 120 bytes \cdot 86.81 \mu s/byte \approx 10.42 ms$$

A stall of $10.42ms$ completely exceeds the entire $10ms$ period of the $100Hz$ loop. The execution frequency collapses, timing jitter skyrockets, and the microcontroller appears to hang.

So, the solution is:

- Keep hihg-frequency executuon tasks free of verbose serial printing

- Downsamples data logging to a dediacted medium-speed task (e.g., $10Hz$)
- At $10Hz$, a $120bytes$ message produces

$$120bytes \cdot 10Hz = 1,200bytes/s$$

  This represents only 10.4% of the UART bus capacity, ensuring the buffer never fills and the CPU never blocks.

Machine-Readable Telemetry: Arduino Serial Plotter Key-Value Formatting

To monitor variables visually without external graphing software, firmware can format serial data using the key-value standard parsed by the **Arduino IDE Serial Plotter**

```
Variable1: Value1, Variable2: Value2, Variable3: Value3\n
```

- Each metric consists of a label and a number separated by a colon (`:`)
- Multiple metrics on the same line are separated by a comma (`,`)
- The line terminates with a standard newline (`\n` via `Serial.println()`)

Example:

```
Uptime_ms: 5200, ExecTime_us: 24, Jitter_us: 12, Waveform: 42.5\n
```

How to test UART with Multimeter

Probing Asynchronous UART Lines with a Digital Multimeter

A Multimeter cannot decode high-speed serial bit patterns (which requires a logic analyzer or oscilloscope), it provides immediate insight into but health

Continuity Mode (Power OFF)

- Verify physical continuity between the ESP32 TX pin and external receiver RX pin
- Confirm zero short-circuit resistance between the TX line and Ground

DC Voltage Mode: IDLE Line State (Power ON, No Data Transmitting)

- Probe the ESP32 TX pin (GPIO 1); must read exactly $+3.3V$ (Mark State)
- A reading of $0.00V$ indicates an unconfigured pin, disabled UART, or short to GND

DC Voltage Mode: Active Transmission Dynamics (Power ON, Active $10Hz$)

- When data transmits, the pin rapidly pulls LOW ($0V$) for Start or data bits and HIGH ($3.3V$) fort Stop or idle bits
- The Multimeter internal analog filter integrates these high-speed pulses into an average DC voltage reading

$$V_{multimeter} = 3.3 \cdot \frac{T_{mark}}{T_{idle}}$$

- For typical ASCII streams, the meter reading dips slightly from $3.3V$ to approximately $3.15V$ to $3.25V$
- A readin below $2.0V$ indicates excessive bus loading, baud mismatch, or data spam

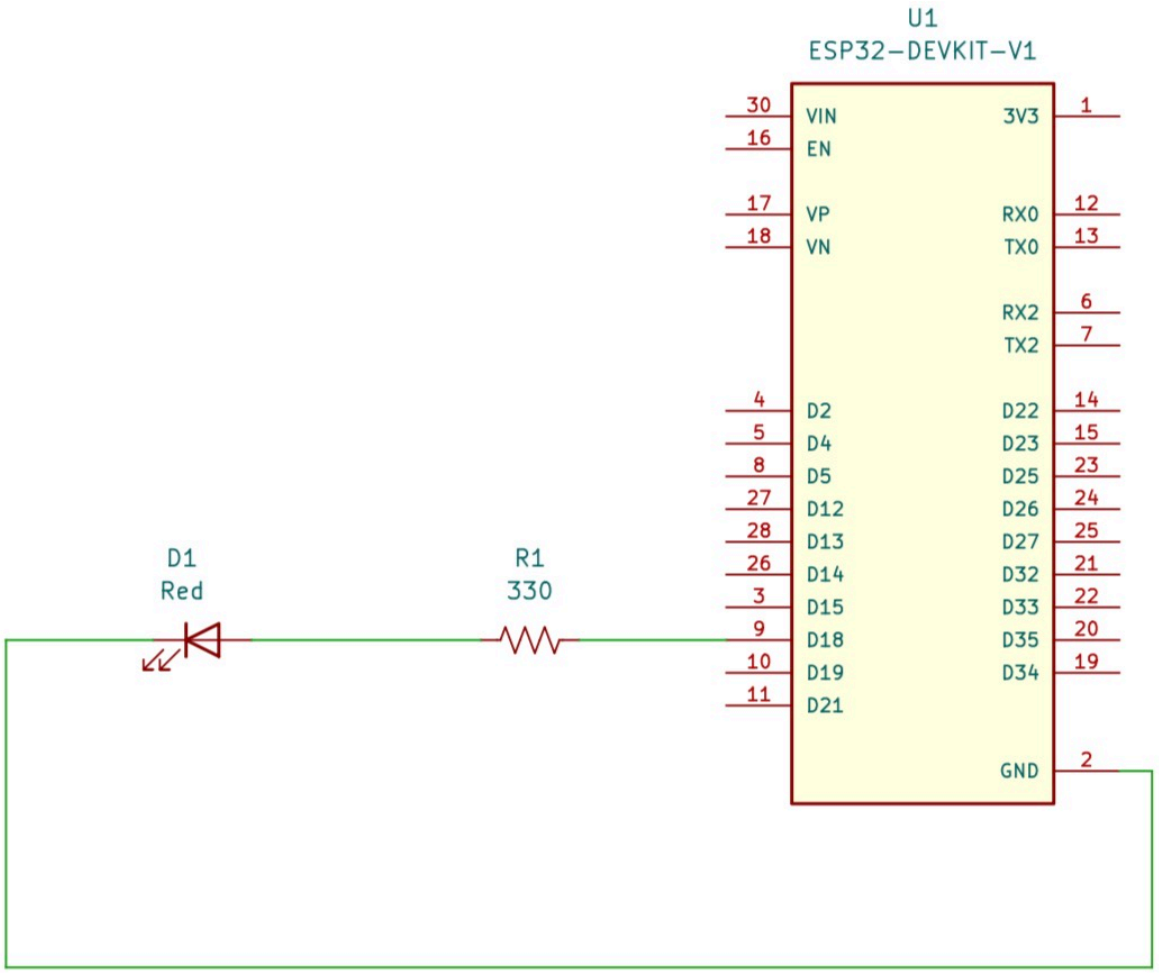
Microcontroller Power Rail Stability Under Processing and Transmission Bursts

High-speed computation and serial transmission cause dynamic current draw transitions from the ESP32 onboard $3.3V$ regulator:

- **Regulated Window:** the $3.3V$ rail should remain stable between $3.25V$ and $3.35V$
- **Brownout Threat:** if breadboard jumper wires are loose or high-resistance, sudden current spikes create instantaneous voltage dips ($\Delta V = I \cdot R$). If the supply dips below $2.8V$, the ESP32 brownout detector triggers an automatic hardware reset

> Probe between the ESP32 `3V3` pin and `GND` using the Multimeter in DC Voltage Mode while the multi-rate sketch is running. Confim that that voltage remains stable without dips

Implementation: Task Scheduler



Pre-Test (Safety)

Before connecting the USB cable:

1. Set the Multimeter to **Continuity Mode**. Probe between the breadboard ground rail and the ESP32 metal USB shield. The meter must emit a continuos beep.
2. Set the Multimeter to **Resistance Mode ($200k\Omega$ range)**. Measure between `3V3` and `GNS`. Confirm that resistance exceeds $100k\Omega$. If resistance is below 10Ω, locate and eliminate the power short before plugging in USB power.

Firmware

Open Arduino IDE, configure the board as **ESP32 Dev Module**, and upload this firmware

```
#include <Arduino.h>

// Hardware pin and LEDC configuration constants
const uint8_t PWM_OUTPUT_PIN = 18;        // PWM output pin
const uint8_t PIN_HEARTBEAT_LED = 4;      // Dedicated status LED pin
```

```
const uint32_t PWM_FREQUENCY = 20000;    // 20 kHz ultrasonic PWM carrier
const uint8_t PWM_RESOLUTION = 8;        // 8-bit duty resolution: 0-255
const uint8_t LEDC_CHANNEL = 0;          // Hardware LEDC channel index

// Scheduler timing intervals
const uint32_t INTERVAL_CORE_LOOP_US = 10000; // 10 ms: 100 Hz motion
task
const uint32_t INTERVAL_TELEMETRY_MS = 100;   // 100 ms: 10 Hz telemetry
task
const uint32_t INTERVAL_HEARTBEAT_MS = 1000;  // 1 s: 1 Hz heartbeat task

// Scheduling epochs
uint32_t last_core_loop_time_us = 0;
uint32_t last_telemetry_time_ms = 0;
uint32_t last_heartbeat_time_ms = 0;

// Performance metrics
uint32_t loop_execution_time_us = 0;     // Most recent motion-task
duration
uint32_t max_execution_time_us = 0;      // Worst observed motion-task
duration
int32_t loop_jitter_us = 0;              // Deviation from the nominal 10
ms period

// PWM state variables
uint8_t current_duty_count = 0;          // Current duty value: 0-255
int8_t ramp_direction = 1;               // +1 accelerates, -1
decelerates

// Heartbeat LED state
bool heartbeat_state = false;

// Initialize the ESP32 LEDC hardware PWM peripheral
bool initHardwarePWM() {
  return ledcAttachChannel(
    PWM_OUTPUT_PIN,
    PWM_FREQUENCY,
    PWM_RESOLUTION,
```

```
    LEDC_CHANNEL
  );
}

// Write an 8-bit duty value to the PWM output
void setHardwarePWM(uint8_t duty_count) {
  ledcWrite(PWM_OUTPUT_PIN, duty_count);
}

// Core motion-control task.
// This task is called at 100 Hz and must remain short and non-blocking.
void task_motion_control() {
  const uint32_t execution_start_us = micros();

  // Apply the current PWM value to the hardware peripheral
  setHardwarePWM(current_duty_count);

  // Reverse the ramp at the upper and lower limits
  if (current_duty_count == 255) {
    ramp_direction = -1; // Begin deceleration
  } else if (current_duty_count == 0) {
    ramp_direction = 1;  // Begin acceleration
  }

  // Update the duty value safely
  current_duty_count = static_cast<uint8_t>(
    static_cast<int16_t>(current_duty_count) + ramp_direction
  );

  // Measure the execution time of this task
  loop_execution_time_us = micros() - execution_start_us;

  // Track the worst observed execution time
  if (loop_execution_time_us > max_execution_time_us) {
    max_execution_time_us = loop_execution_time_us;
  }
}

// Telemetry task.
```

```
// This task is intentionally slower than the motion task so serial
output
// does not interfere with high-frequency control work.
void task_telemetry() {
  const float duty_percent =
    (current_duty_count / 255.0f) * 100.0f;

  // This is the theoretical average voltage on the GPIO pin.
  // It is not the actual motor voltage after an H-bridge driver.
  const float theoretical_voltage =
    (current_duty_count / 255.0f) * 3.3f;

  // Serial.printf() is required because Serial.println() does not
  // interpret printf-style format arguments.
  Serial.printf(
    "PWM_Duty: %u, Duty_pct: %.1f, Theoretical_V: %.3f, "
    "ExecTime_us: %lu, MaxExec_us: %lu, Jitter_us: %ld\n",
    current_duty_count,
    duty_percent,
    theoretical_voltage,
    loop_execution_time_us,
    max_execution_time_us,
    loop_jitter_us
  );
}

// Heartbeat task.
// This task runs once per second and toggles a dedicated LED.
void task_heartbeat() {
  heartbeat_state = !heartbeat_state;

  digitalWrite(
    PIN_HEARTBEAT_LED,
    heartbeat_state ? HIGH : LOW
  );
}

// Arduino initialization function
void setup() {
```

```
// Configure GPIO pins
pinMode(PWM_OUTPUT_PIN, OUTPUT);
pinMode(PIN_HEARTBEAT_LED, OUTPUT);

digitalWrite(PWM_OUTPUT_PIN, LOW);
digitalWrite(PIN_HEARTBEAT_LED, LOW);

// Initialize the USB serial console
Serial.begin(115200);
delay(500); // Allow the serial interface to initialize

Serial.println("SCHEDULER");

// Initialize the hardware PWM peripheral
if (!initHardwarePWM()) {
  Serial.println("[FATAL ERROR] Failed to initialize LEDC hardware");

  // Stop here if PWM initialization fails
  while (true) {
    delay(1000);
  }
}

// Ensure the PWM output starts at zero duty
setHardwarePWM(0);

Serial.printf(
  "[STATUS] Hardware LEDC Peripheral Initialized\n"
  "GPIO Pin: %u\n"
  "Frequency: %lu Hz\n"
  "Resolution: %u-bit\n",
  PWM_OUTPUT_PIN,
  PWM_FREQUENCY,
  PWM_RESOLUTION
);

// Initialize all scheduler epochs from the same time reference
const uint32_t start_time_us = micros();
const uint32_t start_time_ms = millis();
```

```
  last_core_loop_time_us = start_time_us;
  last_telemetry_time_ms = start_time_ms;
  last_heartbeat_time_ms = start_time_ms;

  Serial.println("[STATUS] Non-blocking scheduler started");
}

// Main Arduino loop.
// Each task is checked independently and returns quickly.
void loop() {
  const uint32_t current_time_us = micros();
  const uint32_t current_time_ms = millis();

  // 100 Hz motion-control task
  if ((uint32_t)(current_time_us - last_core_loop_time_us) >=
      INTERVAL_CORE_LOOP_US) {

    // Measure the actual time since the previous scheduled slot
    const uint32_t actual_period_us =
      current_time_us - last_core_loop_time_us;

    // Jitter is the difference from the nominal 10 ms period
    loop_jitter_us =
      static_cast<int32_t>(actual_period_us) -
      static_cast<int32_t>(INTERVAL_CORE_LOOP_US);

    // Preserve the absolute scheduling grid.
    // Unsigned arithmetic also handles micros() rollover correctly.
    last_core_loop_time_us += INTERVAL_CORE_LOOP_US;

    task_motion_control();
  }

  // 10 Hz telemetry task
  if ((uint32_t)(current_time_ms - last_telemetry_time_ms) >=
      INTERVAL_TELEMETRY_MS) {

    // Preserve the absolute scheduling grid
```

```
    last_telemetry_time_ms += INTERVAL_TELEMETRY_MS;

    task_telemetry();
  }

  // 1 Hz heartbeat task
  if ((uint32_t)(current_time_ms - last_heartbeat_time_ms) >=
      INTERVAL_HEARTBEAT_MS) {

    // Preserve the absolute scheduling grid
    last_heartbeat_time_ms += INTERVAL_HEARTBEAT_MS;

    task_heartbeat();
  }
}
```

Test

1. Upload `robot_main.ino` to the ESP32.
2. Open the **Arduino Serial Monitor** (`115200` baud). Confirm clean lines of telemetry outputting exactly 10 times per second ($10\ \mathrm{Hz}$).
3. Open **Tools > Serial Plotter** (`Ctrl` + `Shift` + `L`). Verify that 4 distinct curves appear:
   - **`PWM_Duty`** : A smooth triangle wave ramping continuously between 0 and 255 counts
   - **`Theoretical_V`** : Theoretical average output voltage rising and falling smoothly between $0.00\mathrm{V}$ and $3.30\mathrm{V}$.
   - **`ExecTime_us`** : Baseline compute time of the 100 Hz task (typically $2\ \mu\mathrm{s}$ to $15\ \mu\mathrm{s}$).
   - * `Jitter_us` **: Variation around the nominal $10,000\ \mu\mathrm{s}$ period (typically $< \pm 50\ \mu\mathrm{s}$).
4. Switch the Multimeter to **DC Voltage Mode ($20\mathrm{V}$ range)**:
   - **Probe GPIO 18 (PWM Output / Test Pin)**: As `PWM_Duty` ramps, the DMM internal low-pass filter integrates the $20\ \mathrm{kHz}$ pulse train, displaying a smooth DC voltage rise from $0.00\mathrm{V}$ **to** $3.30\mathrm{V}$ (and back down), tracking `Theoretical_V` within $\pm 0.05\mathrm{V}$.
   - **Probe GPIO 2 / GPIO 18 (Heartbeat LED)**: The voltage alternates cleanly between $0.00\mathrm{V}$ and $3.30\mathrm{V}$ once per second ($1\ \mathrm{Hz}$).
   - **Probe GPIO 1 (TX Pin)**: Measure resting voltage during active $10\ \mathrm{Hz}$ telemetry stream; the meter should display between $3.15\mathrm{V}$ **and** $3.28\mathrm{V}$, confirming brief logic transitions during transmission.
   - **Probe `3V3` Rail**: Confirm that the voltage regulator holds rock-solid between $3.25\mathrm{V}$ and $3.35\mathrm{V}$

Debugging

Serial Monitor Shows Garbled Symbols or Question Marks

- The baud rate setting in the bottom-right corner of the Serial Monitor does not match the firmware's `Serial.begin(115200)` configuration. Change the monitor setting to `115200 baud`

Heartbeat LED Blinks Erratically or Freezes

- Check whther any function inside `loop()` takes longer than $1000ms$ or contains a blocking `delay()`
- Check whether `Serial.print()` is being called inside a fast loop without downsampling, which fills the transmit buffer and freezes

Execution Time (T_{exec}) Spikes Anormally

- high-level formatting functions such as `String` concatenation or floating-point division take significantly more FPU cycles than integer arithmetic. Use fixed buffers and minimal formatting in fast tasks

References

- Making Embedded Systems: Design Patterns for Great Software by Elecia White
- Computer Systems: A Programmer's Perspective by Randal E. Bryant and David R. O'Hallaron
- ESP32 Arduino Core Documentation: Serial (UART) by Espressif System
- General Purpose Timer (GPTimer) - ESP32 Programming Guide by Espressif Systems
- Arduino Core for the ESP32 by Espressif Systems
- The Art of Electronics: Student Manual by Thomas C. Hayes and paul Horowitz